Step 1 — The Report Shape, the Executor, and the Compiler — Solution

> Back to the step · Hint 1 · Hint 2

Story: this builds the engine — run each research step, collect findings, cite their sources — essentially the whole project minus the model, proven with a fake search standing in. **If not:** citation numbering and source de-duplication are genuinely fiddly logic; debugging them *and* real model behavior at the same time in Step 3 would be miserable.

Setup files

`requirements.txt`: `langchain-openai`, `langchain-community`, `langchain-text-splitters`, `chromadb`, `openai`, `pydantic`, `python-dotenv`. `.env.example`: `OPENAI_API_KEY=sk-your-key-here` / `LOG_LEVEL=INFO`. `.gitignore`: `.env`, `.venv/`, `__pycache__/`, `*.pyc`, `chroma_db/`. Every folder under `src/` gets an empty `__init__.py`. `LICENSE` is the MIT text.

Settings, errors, logging, model

```
# src/exceptions.py
class MissingConfigError(Exception):
    """A required setting is missing from the environment."""
    pass
```

```
# src/config.py
import os
from dataclasses import dataclass
from dotenv import load_dotenv
from src.exceptions import MissingConfigError

@dataclass
class Config:
    openai_api_key: str
    log_level: str

def require_env(key):
```

```
    value = os.getenv(key)
    if value is None or value == "":
        raise MissingConfigError(
            f"Required environment variable is missing: {key}"
        )
    return value

def load_config():
    load_dotenv()
    return Config(
        openai_api_key=require_env("OPENAI_API_KEY"),
        log_level=os.getenv("LOG_LEVEL", "INFO"),
    )
```

```
# src/utils/logger.py
import logging
import os

def get_logger(name):
    logger = logging.getLogger(name)
    if not logger.handlers:
        level_name = os.getenv("LOG_LEVEL", "INFO")
        level = getattr(logging, level_name.upper(), logging.INFO)
        handler = logging.StreamHandler()
        handler.setLevel(level)
        log_format = "%(asctime)s %(name)s %(levelname)s %(message)s"
        handler.setFormatter(logging.Formatter(log_format))
        logger.addHandler(handler)
        logger.setLevel(level)
    return logger
```

```
# src/models/llm.py
from langchain_openai import ChatOpenAI

def get_model(config):
    return ChatOpenAI(
        model="gpt-4o-mini", temperature=0, api_key=config.openai_api_key
    )
```

The typed shapes

```
# src/schemas.py
# SubQuestions is the planner's output; Finding is one researched sub-question
```

```
# with its source; Report is the compiled result.
from typing import List

from pydantic import BaseModel

class SubQuestions(BaseModel):
    sub_questions: List[str] = []

class Finding(BaseModel):
    sub_question: str
    answer: str
    source: str

class Report(BaseModel):
    question: str
    findings: List[Finding] = []
```

The executor

```
# src/executor.py
# Run the search() for each sub-question and collect a Finding. search() is
# passed in - a RAG lookup in the real pipeline, a fake one in tests - so this
# engine needs no model or embeddings to test.
from src.schemas import Finding

def execute_plan(sub_questions, search):
    findings = []
    for sub_question in sub_questions:
        text, source = search(sub_question)
        findings.append(Finding(
            sub_question=sub_question, answer=text, source=source,
        ))
    return findings
```

The compiler and the citation formatter

```
# src/report.py
# Fold the findings into a Report, and format it with numbered citations.
# Sources are numbered in first-seen order and de-duplicated, so two findings
# from the same document share one citation number.
from src.schemas import Report
```

```
def compile_report(question, findings):
    return Report(question=question, findings=findings)

def format_report(report):
    sources = []
    for finding in report.findings:
        if finding.source not in sources:
            sources.append(finding.source)

    lines = [f"Research: {report.question}", ""]
    for finding in report.findings:
        number = sources.index(finding.source) + 1
        lines.append(f"- {finding.sub_question}")
        lines.append(f"  {finding.answer} [{number}]")
    lines.append("")
    lines.append("Sources:")
    index = 1
    for source in sources:
        lines.append(f"  [{index}] {source}")
        index += 1
    return "\n".join(lines)
```

The tests (no model, no embeddings)

```
# tests/unit/test_config.py
# Run from the repo root: python -m tests.unit.test_config
import os

from src.config import load_config, require_env
from src.exceptions import MissingConfigError

TEST_SETTING = "RESEARCHPILOT_TEST_SETTING"

def test_missing_setting():
    os.environ.pop(TEST_SETTING, None)
    result = "WRONG"
    try:
        require_env(TEST_SETTING)
    except MissingConfigError:
        result = "OK"
    print(f"case 1 (missing setting) -> {result}")

def test_config_loads():
    os.environ["OPENAI_API_KEY"] = "sk-test-123"
    config = load_config()
```

```
    result = "OK"
    if config.openai_api_key != "sk-test-123":
        result = "WRONG"
    print(f"case 2 (config loads)    -> {result}")

if __name__ == "__main__":
    test_missing_setting()
    test_config_loads()
```

```
# tests/unit/test_report.py
# Run from the repo root: python -m tests.unit.test_report
from src.executor import execute_plan
from src.report import compile_report, format_report

# a fake search: a distinctive keyword -> (answer, source)
TABLE = [
    ("rag", ("RAG retrieves then generates.", "rag.md")),
    ("agent", ("An agent is an LLM in a loop.", "agents.md")),
    ("embedding", ("Embeddings turn text into vectors.", "rag.md")),
]

def fake_search(query):
    lowered = query.lower()
    for key, value in TABLE:
        if key in lowered:
            return value
    return ("No information found.", "none")

def build_report():
    plan = ["What is RAG?", "What is an agent?", "What is an embedding?"]
    findings = execute_plan(plan, fake_search)
    return compile_report("RAG and agents", findings)

def test_findings_appear():
    text = format_report(build_report())
    result = "OK"
    if "An agent is an LLM in a loop." not in text:
        result = "WRONG"
    print(f"case 1 (findings in report)      -> {result}")

def test_citations_dedup_sources():
    text = format_report(build_report())
    result = "OK"
    # rag.md and embedding share source rag.md -> both cite [1]
    if text.count("[1]") < 2:
        result = "WRONG"
```

```
    if "[1] rag.md" not in text or "[2] agents.md" not in text:
        result = "WRONG"
    print(f"case 2 (citations numbered/dedup) -> {result}")

if __name__ == "__main__":
    test_findings_appear()
    test_citations_dedup_sources()
```

Expected output:

```
case 1 (missing setting) -> OK
case 2 (config loads)    -> OK
```

```
case 1 (findings in report)      -> OK
case 2 (citations numbered/dedup) -> OK
```

The full report for the fake plan looks like this — note "embedding" and "RAG" both cite `[1]` because they share `rag.md`:

```
Research: RAG and agents

- What is RAG?
  RAG retrieves then generates. [1]
- What is an agent?
  An agent is an LLM in a loop. [2]
- What is an embedding?
  Embeddings turn text into vectors. [1]

Sources:
  [1] rag.md
  [2] agents.md
```

The citation + de-duplication logic is the real work here, and it's proven with made-up data — so Step 3 only has to feed it real sub-questions and real sources.